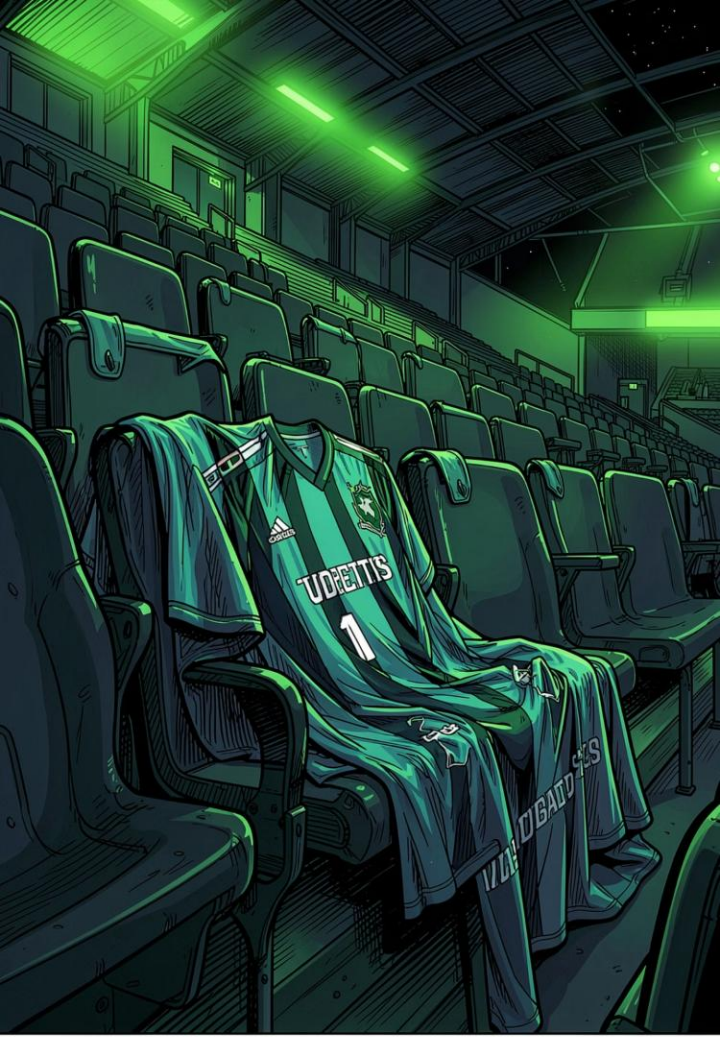


Eleven Store — Football Beyond Match Day

Football fans wear jerseys only during matches. Jerseys are expensive, impractical for daily wear, and football-inspired lifestyle apparel is scarce — leaving fans without a way to express their passion off the pitch.

> "Enable football lovers to express their passion beyond match days."
>
> -S.Charaan

CUSTOMER RESEARCH

What Football Fans Actually Want

Research Methods

- Customer interviews with active fans
- Football community discussions
- Social media sentiment analysis

Key Findings

- Fans want **affordable** football-inspired clothing
- Style matters — **daily wear** is the goal
- Stronger connection to **football culture** desired

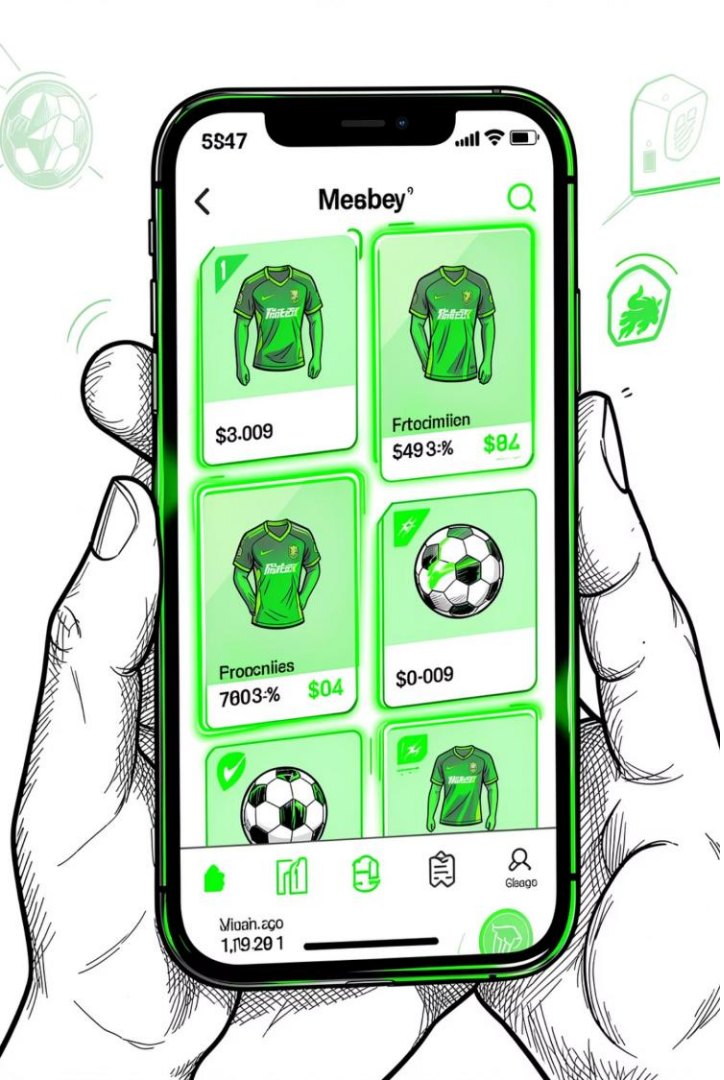
User Persona

Goals: Express football identity daily through stylish, affordable apparel
Pain Points: Jerseys too expensive and impractical for everyday wear
Motivations: Belonging to football culture beyond the 90 minutes

USER EXPERIENCE

The Fan Journey: From Discovery to Loyalty

Each stage balances **user goals** (ease, identity, trust) with **business goals** (conversion, retention, brand affinity). The WhatsApp checkout creates a personal touchpoint that reinforces community over transaction.



MVP SCOPE

What We Delivered in MVP

Landing Page

Football-themed branding, About section, and contact details to establish trust and identity

Authentication

Full signup, login, and logout flow with session management

Shopping Experience

Product listing, cart management, and streamlined checkout flow

Deployment

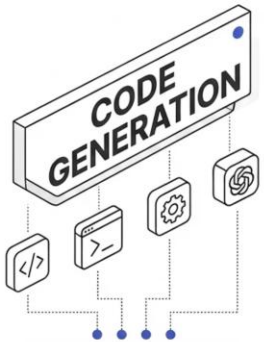
Version-controlled on GitHub, live on Render — production-ready from day one

We focused on delivering a Minimum Viable Product (MVP) before scaling — validating demand before optimizing infrastructure.

TRADE-OFF #1

IDE Development vs. Pure Vibe Coding

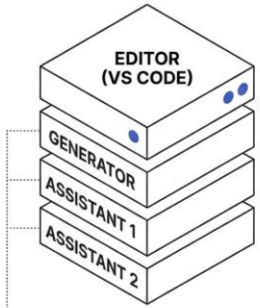
OPTION A:
PURE VIBE CODING



Fast Generation.
Limited Control.

Hard to Debug.
No Code Ownership.

OPTION B:
VS CODE + ANTIGRAVITY
+ CLAUDE + CHATGPT



• Better Debugging.
Full Code Ownership.
Easier Customization.
Professional Workflow.
Deeper Architecture
Understanding.

Decision: Option B

We chose **Antigravity + AI tools** over pure vibe coding. The slight slowdown in development was worth gaining full code ownership, better debugging, and a professional workflow.

> "The objective was not only to generate code but also to understand, customize, maintain, and deploy a real-world application."

TRADE-OFF #2

React.js vs. Traditional HTML/CSS/JS

Why React.js Won

*Component Reusability

Build once, reuse everywhere — ProductCard, Navbar, CartItem

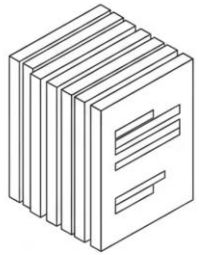
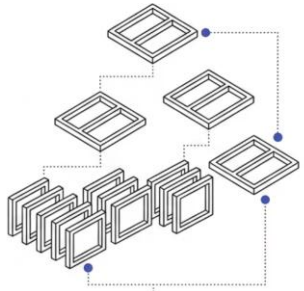
*Scalability

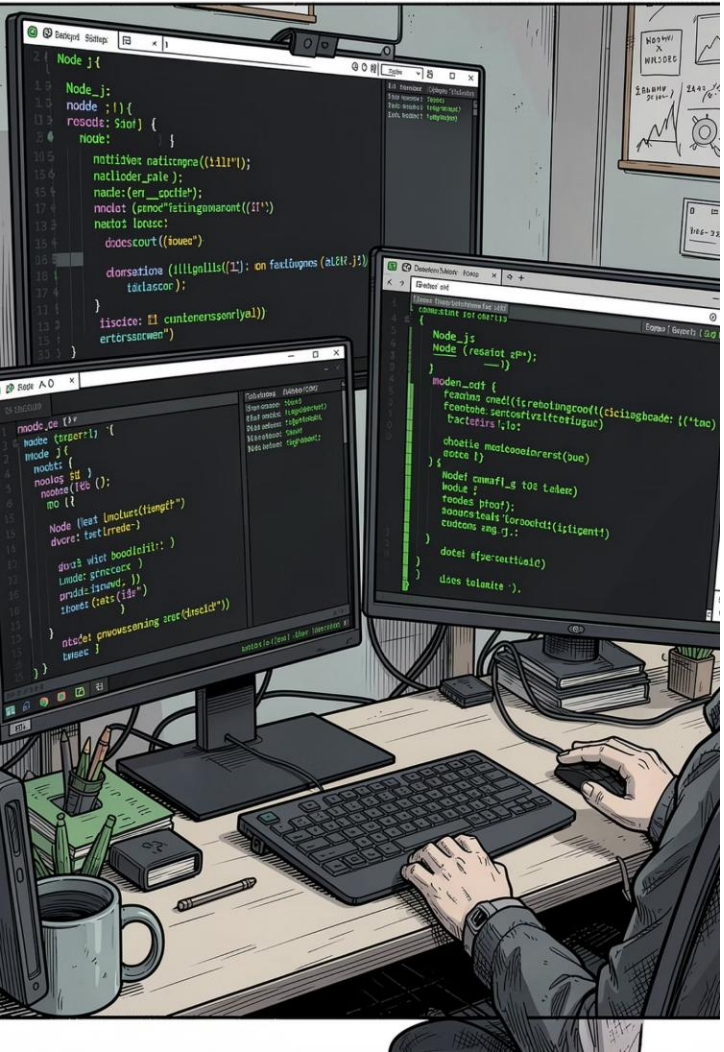
New features plug in without rewriting the foundation

*State Management

Handles cart, auth, and product state cleanly

"React allows faster feature expansion as the platform grows."

HTML+CSS+JS	React.js
	
Simple setup No learning curve	Learning curve More setup
Hard to scale Repetitive code Poor state management	Component reusability Scalability Better state management Strong industry adoption



TRADE-OFF #3

Node.js + Express vs. Enterprise Frameworks

Spring Boot

Enterprise-grade, heavy structure, Java ecosystem — overkill for MVP

Django

Batteries-included Python framework — great but adds language context-switch

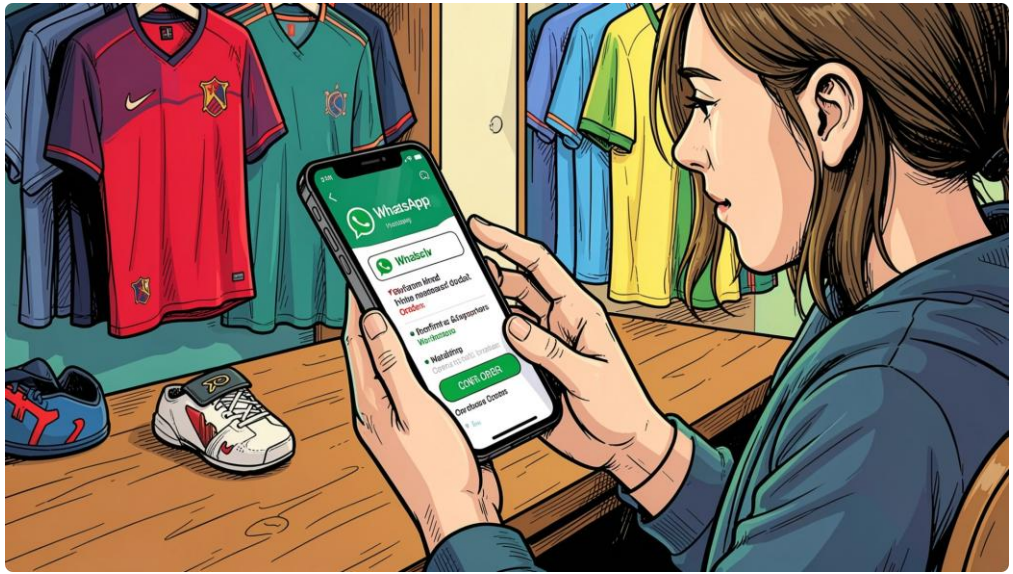
✓ Node.js + Express

JavaScript end-to-end, lightweight, fastest path from idea to deployment

Trade-off acknowledged: Less enterprise structure than Spring Boot — but **speed of execution was prioritized over enterprise complexity.**

TRADE-OFF #4

WhatsApp Checkout vs. Payment Gateway



Why WhatsApp for MVP

⚡ Faster Launch

Zero payment gateway setup — live in hours, not days

💬 Direct Interaction

Personal touch builds trust and early customer relationships

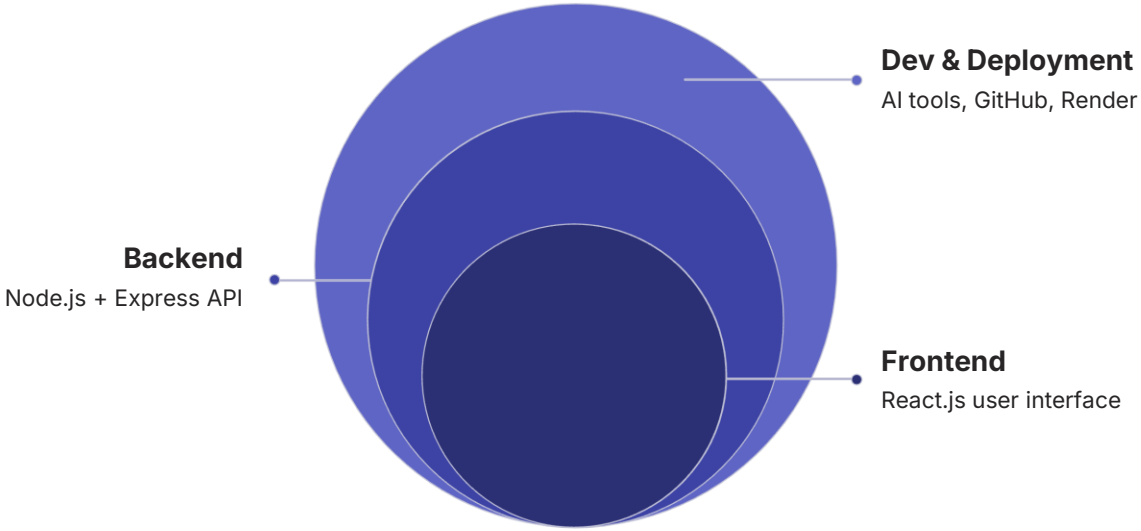
🎯 Validate First

Demand validation mattered more than transaction optimization

⚠ **Trade-off:** Manual order processing and limited scalability — addressed in Phase 2 with Razorpay.

ARCHITECTURE

Technology Stack & System Architecture



The architecture prioritizes simplicity and speed: a React frontend communicates with a Node.js + Express backend, all developed with AI assistance and deployed on Render.

Frontend

React.js — component-driven UI with responsive design

Backend

Node.js + Express.js — RESTful API, lightweight and fast

AI Tools

Antigravity, Claude, ChatGPT — code generation and debugging

DevOps

GitHub for version control, Render for cloud hosting

ROADMAP

Learnings & What's Next

1

Product Learnings

Customer interviews, persona design, journey mapping, MVP thinking, and trade-off analysis

2

Technical Learnings

React development, backend integration, GitHub workflow, and cloud deployment on Render

3

Phase 2

MongoDB database, secure authentication, and admin dashboard

4

Phase 3

Razorpay payments, order tracking, and analytics dashboard

"Eleven Store demonstrates how Product Thinking, AI-Assisted Development, Full Stack Engineering, and Customer-Centric Design can be combined to build a real-world digital product."